Δ DELTA FILINGS

Corporate Governance Intelligence Platform

Technical Architecture Document

Version 2.0 — April 2026

Table of Contents

1. Overview

Delta Filings v2 is a multi-service web platform built on a containerised stack. This Technical Architecture Document describes the services, data flows, deployment topology and non-functional guarantees of the system. It pairs with the BRD and FRD to close the loop between business intent and runtime design.

2. Architectural Principles

- Separation of concerns: UI, API, ingestion and data are separate services.
- Statelessness: application tiers are stateless and horizontally scalable.
- Cache by default: Redis sits in front of expensive reads and external calls.
- Reproducibility: any environment can be brought up from a single Docker Compose file.
- Observability before features: structured logs and metrics accompany every new endpoint.
- Security by default: authenticated by default, explicit allow-lists for public endpoints.

3. System Context

Delta Filings interacts with three external categories of systems: statutory data sources (BSE, NSE, MCA), identity and analytics providers (Supabase, OpenAI), and end-user browsers. The platform itself is an internally-structured monolith of four principal services.

External System	Direction	Protocol	Purpose
BSE announcements feed	Inbound pull	HTTPS / HTML	Filing ingestion.
NSE announcements feed	Inbound pull	HTTPS / JSON	Filing ingestion.
MCA director portal	Inbound pull	HTTPS / HTML	Director records for DIN lookup.
Supabase	Bidirectional	HTTPS / JWT	Authentication and user store.
OpenAI	Outbound	HTTPS / REST	Filing summarisation and classification.
Transactional email provider	Outbound	HTTPS / REST	Alerts and digests.

4. Logical Components

Component	Runtime	Responsibilities
Frontend	Next.js 15 on Node 20	SSR, hydration, UI, session cookie handling.
API Gateway / Backend	FastAPI (Python 3.12)	REST endpoints, authz, business logic.
Ingestion Workers	Python async tasks	Scrape BSE/NSE/MCA, normalise, persist.
Summary Worker	Python async tasks	Fetch filings from queue, call OpenAI, persist summary.
Alerts Worker	Python async tasks	Watchlist alert generation and dispatch.
Postgres	PostgreSQL 16	System of record.
Redis	Redis 7	Cache, dedup, job queue (streams).
Object Storage	S3-compatible	Evidence uploads, exported PDFs.

5. Deployment Topology

5.1 Development

All services brought up locally via Docker Compose. Bind mounts enable live reload for frontend and backend.

```
services:   postgres:  postgres:16-alpine        (port 5432)   redis:
redis:7-alpine            (port 6379)   backend:   fastapi + uvicorn --
reload (port 8000)   frontend:  next dev                  (port 3000)
```

5.2 Production

- Frontend: Next.js standalone build served behind a CDN.
- Backend: container orchestrator (Fly/Render/ECS) with at least 2 replicas behind a load balancer.
- Workers: separate deployment per worker type; auto-scaled by queue depth.
- Postgres: managed instance with HA + daily backups, retained 30 days.
- Redis: managed instance with eviction policy 'allkeys-lru' and persistence disabled (cache-only usage).
- Secrets: managed vault; no secrets in images or environment-less config.

6. Data Model (Simplified)

The table below captures the core relational schema. Full DDL lives in the Alembic migration history.

Table	Key Columns	Notes
user_profiles	id (uuid, PK), email, display_name	Mirrors Supabase user; 1:1 on id.
companies	id, symbol, isin, cin, sector, listed_on	Unique(symbol, exchange).
filings	id, company_id, category, published_at, source_url, summary, materiality	Indexed on (company_id, published_at desc).
directors	din (PK), name, dob_year, is_disqualified, updated_at	
director_associations	director_din, company_id, role, start_date, end_date	Composite PK.
watchlists	id, user_id, name, created_at	Unique(user_id, name).
watchlist_members	watchlist_id, company_id	Composite PK.
obligations	id, statute, title, frequency, applicability_json	
compliance_runs	id, obligation_id, company_id, status, due_date, assignee_id, evidence_json, hash	Immutable once status = 'complete'.
audit_log	id, actor_id, action, target_type, target_id, created_at, payload_json	Append-only.

7. API Surface

All endpoints are prefixed with /api. Authenticated endpoints require a Supabase JWT either as a cookie or bearer token.

Method	Path	Purpose
GET	/api/health	Liveness probe.
GET	/api/filings	List filings with filters.
GET	/api/filings/{id}	Filing detail.
GET	/api/directors/{din}	Director lookup.
POST	/api/compare	Create a director comparison.
GET	/api/watchlists	List the user's watchlists.
POST	/api/watchlists	Create a watchlist.
GET	/api/compliance/obligations	List obligations for a company.
POST	/api/compliance/runs	Create a compliance run.
GET	/api/compliance/runs/{id}/export	Export run as PDF.

8. Ingestion Pipeline

Each ingestion worker follows the same pipeline pattern: fetch, normalise, dedup, persist, enqueue downstream work.

```
[Scheduler] --tick every 60s during market hours--> [Scraper]  --parse
HTML/JSON--> [Normaliser] [Normaliser] --sha256(source_url+timestamp)-->
[Dedup via Redis] [Dedup miss] --insert--> [Postgres filings]
--enqueue--> [Redis stream: summary_jobs] [Summary Worker] --consume-->
[OpenAI chat.completions]                  --update filings.summary,
filings.materiality-->                 --enqueue--> [Redis stream:
alert_jobs]
```

9. Security

9.1 AuthN / AuthZ

- Supabase issues RS256 JWTs; backend verifies signature using the Supabase public JWKS.
- Session cookie is httpOnly, Secure, SameSite=Lax; Expiry 60 minutes with refresh rotation.
- Per-endpoint scope enforcement (public / user / compliance-write / admin).

9.2 Network

- TLS 1.2+ for all external traffic; internal service-to-service over private network.
- CORS allow-list is environment-configured; no wildcards in production.
- Rate limits at the API layer keyed by user_id and IP.

9.3 Data

- No card numbers, bank accounts, government IDs stored.
- Evidence uploads scanned for MIME validity; size cap at 25 MB per file.
- Backups encrypted at rest; access gated behind on-call role.

10. Observability

- Logs: structured JSON, trace-id propagated across services, shipped to a central log store.
- Metrics: Prometheus-compatible, per-endpoint latency histograms and error counters.
- Tracing: OpenTelemetry spans across frontend-fetch, backend-request and worker-job.
- Synthetic probes: /api/health, /dashboard and /din every 60s.
- Alerts: Pager duty routes for Sev-1 with 15-minute acknowledgement SLO.

11. Performance Targets

Surface	Metric	Target
Dashboard	Time to interactive (p95, 4G)	< 2.5s
API	Latency (p95, authenticated)	< 400ms
DIN lookup	Latency (p95, cache hit)	< 250ms
Ingestion	Lag from source publication (p95)	< 15m
Alerts	Dispatch latency after ingestion (p95)	< 2m

12. Reliability and Backups

- RPO: 24 hours. Backups run daily with 30-day retention.
- RTO: 4 hours for Sev-1 incidents during market hours.
- Monthly restore-drill on a non-production environment.
- Stateless services recovered by redeploying last healthy image.

13. Trade-offs and Alternatives Considered

Decision	Chosen	Alternative	Reason
Backend framework	FastAPI	Django REST, Flask	Async-first, strong typing, light footprint.
Frontend framework	Next.js 15 App Router	Remix, Vite + React Router	Mature SSR, strong ecosystem, easy deploy.
Primary DB	PostgreSQL	MySQL, DynamoDB	Rich SQL, extensions (JSONB, pg_trgm).
Queue	Redis Streams	RabbitMQ, SQS	Reuses cache infra; adequate throughput at v2 scale.
Auth	Supabase	Auth0, custom	Fast onboarding; JWT model interoperable.

14. Roadmap Dependencies

- M2 blocks on ingestion pipeline + summary worker.
- M3 blocks on director scraper hardening + curated snapshot pipeline.
- M4 blocks on object storage integration + PDF export service.
- M5 blocks on transactional email integration and preference UI.
- M7 blocks on billing (Stripe/Razorpay) integration and SSO support.

15. Open Questions

- Do we introduce a search index (OpenSearch) in v2 or defer to v3?
- Will compliance runs require digital signatures for audit-level legal weight?
- Residency — which managed Postgres provider has a true Indian region option by M6?